

Mind the Semantic Process Gap: Discovery and Application of Object Role Ontologies from Object-Centric Event Logs

No Author Given

No Institute Given

Abstract. The object-centric (OC) paradigm has emerged as a powerful lens for discovering object interactions throughout a process. Indeed, since a process’s behaviour will be influenced by real-world constraints on and relations between objects (e.g., purchase orders, their associated items and quotes for these items), there is a clear need for understanding the object aspect of an event log. Existing OC process mining methods use objects as an additional dimension to further describe control flow. We instead present an approach to discover an Object Role Ontology (ORO) from an event log, which specifically focuses on semantic object roles, relations and constraints based on object synchronizations and temporal event orderings. Notably, this object-focused approach allows aligning discovered object roles with established ontological categories. In doing so, event log data can be aligned with real-world ontologies, and their semantics can be leveraged for improved understanding, compliance checking, and error correction. We formalize object role ontologies using the *ALCQI* description logic, providing a unified, computationally tractable framework for object-centric process analysis. We situate our work within the state of the art on object-centric process mining. We demonstrate the practical applicability of this methodology through a python-based virtual knowledge graph implementation, evaluated on real-world benchmark event logs.

Keywords: Process Mining · Applied Ontology · Object-Centric Event Logs.

1 Introduction

Since the advent of process mining, event logs have relied on three key attributes: event type (or activity), timestamp, and a “case” identifier [23] to identify distinct instances of process execution. However, this ‘one case to rule them all’ approach has been shown to have issues in accurately capturing true process semantics [9]. For instance, when a single supplier order contains multiple items serving distinct customer orders, any single case identifier will obscure the reality of the process and hinders analysis. Consequently, the field is increasingly shifting away from this rigid case-centric paradigm toward an *object*-centric one [9, 24].

OCEL 2.0 [3] is a prominent emergent standard for modelling object-centric process data. One of the key pieces of data OCEL 2.0 permits is explicit object-to-object (O2O) relations. These relations link object instances (critically, not object types) within the datastore and may optionally include a text string, known as a “qualifier,” to label the connection. Whether qualified or not, the flexibility of these isolated relations allow only a shallow description of how objects are related in a process. In fact, this reliance on relatively simple object relations neglects a fundamental reality: interactions between objects, through process events, are characterized and ontologically constrained by foundational and domain realities. For instance, items placed into a shipping box will remain physically collocated with that box until an un-packaging event occurs [22]. Similar constraints hold true for *any* physical containment relation (I.e., based on foundational knowledge). These ontologically-grounded process semantics cannot be adequately captured by attribute-value pairs alone.

Rather than relying on O2O relations, one can instead build on the “source” of process object interactions, namely their co-participation within events, commonly called “synchronizations” [10, 14, 7]. Nevertheless, synchronizations only serve as an initial characterization, and an ontology-aware approach is needed that formally defines distinct behaviours based on foundational knowledge. For example, consider a forklift unloading a shipping container from a truck: synchronization alone cannot adequately describe the distinct ontological roles they play, such as an agent performing work, an object being manipulated, and a resource being utilized. Synchronizations can be made more meaningful by interpreting properties of participating objects across a log, including their temporal ordering and multiplicities. By abstracting “synchronization behaviours” in this way, we can derive ontologically grounded *roles* that objects play in a process, such as creation, containment, resource utilization, transformation, and closure. Analysis and abstraction of these role behaviours then form an *Object Role Ontology* (ORO) directly from process behaviour, enabling formal reasoning about the classes of objects and events that govern the underlying process.

First, we provide a methodology to extract and abstract implied object interactions from object-centric event logs into an object role ontology (ORO). Beyond bottom-up discovery, this allows practitioners to use a top-down approach to verify domain knowledge against raw data and check the conformance of new logs. Second, we formalize these interactions using *ALCQI* description logic. These logic and conceptual modelling choices facilitates alignment with Semantic Web technologies [6, 19] and with established foundational ontologies [12, 4, 18], while balancing expresiveness and tractability [8]. It further represents a computable and re-usable artifact as the endpoint of the formalization. Finally, we offer a preliminary implementation and tool support via an open-source Python library based on virtual knowledge graphs.

The paper is structured as follows: Section 2 reviews related work in object-centric process mining and applied knowledge modelling. Section 3 formalizes the object interaction graph (OIG). Section 4 introduces structural aggregations of the OIG. Section 5 details the *ALCQI* formalization of object roles and ontolog-

ical constraints. Section 6 presents the implementation and empirical validation. Finally, Section 7 discusses key takeaways and directions for future work.

2 Related Work

The object-centric paradigm in process mining has significantly advanced the analysis of complex object interactions beyond the limitations of a singular case notion. However, a critical ‘(onto)logical divide’ remains. While existing works provide utility for querying and discovering constraints on object-centric control flow [15, 14, 24, 7], they prioritize data-driven flexibility over ontological grounding [13], often reducing distinct ontological commitments to annotated control flow constraints. Preliminary efforts to bridge this gap either lack a native object-centric focus [17, 16] or struggle to operationalize formal ontologies for large-scale, data-driven process mining [13].

Object-Centric Process Constraints and Discovery Process mining is built on data-driven discovery and enforcement of process behaviour. Recent work has advanced the ability to query patterns of behaviour based on object and event relationships in Object-Centric Event Logs (OCEL) [15, 25] and to discover declarative-style process models [14] incorporating object information. Similarly, Object-Centric Behavioural Constraints (OCBC) [24] presents a dual mode modelling framework for process constraints in terms of objects and activities. However, while these approaches facilitate more sophisticated analysis, they lack an ontological grounding. Objects are treated as an additional layer of domain-level attributes to constrain control flow, rather than as entities with inherent behavioural semantics. For instance, labels like “create” or “update” have no inherent connection to their foundational ontological interpretation. Furthermore, these methods apply largely at the trace level, lacking the expressiveness to precisely constrain objects *across* process occurrences, as is the case for our approach (e.g., global reusability). This failure to capture unique object roles creates a disconnect from foundational process knowledge, hindering both verification and ontology alignment. Critically, these methods lack abstraction mechanisms to natively handle foundational concepts like object creation, containment, or parthood. Instead, practitioners must use ad-hoc encodings within bespoke constraint or query vocabularies. Furthermore, a methodological gap persists in these object-centric approaches: practical implementations (e.g., in Rust [14, 15]) are disconnected from their mathematical formalizations. This lack of formal verification ultimately undermines the transparency and verifiability of their ontological commitments.

Event Knowledge Graphs Event Knowledge Graphs (EKGs) [7] enable the representation of distinct process perspectives based on objects using labelled property graphs. While efficient for structuring complex multi-dimensional relational data, they lack the formal description logic grounding (e.g., RDF [19] and OWL

[6]) that enables transparent and verifiable reasoning, as in Semantic Web knowledge graphs. Consequently, EKGs rely on implicit ontological commitments embedded within Neo4j Cypher queries rather than formal logic, limiting their transparency and interoperability. Recent refinements introduce JSON-encoded semantic headers [22] to map object-centric event data to a PG-schema [1], which however lacks an inference mechanism. Without an underlying formal logic, challenges remain for logical verification, open-world reasoning, and ontological alignment. This (onto)logical divide remains an issue, reducing meaningful object interactions to ‘flattened’ graph properties rather than ontologically grounded relationships.

Formal Ontologies for Process Mining Recent efforts have attempted to apply formal ontologies to process mining but face challenges with object-centricity and computational scalability. In [17], the authors leverage “knowledge patterns” [16] and extend ontology-based data access (OBDA) methods to model implicit ontological commitments in process mining workflows as *data theories*, expressed in terms of a foundational process ontology. However, this architecture lacks support for object-centric data and requires further validation for reasoning with large-scale datasets.

Hooshyar et al. [13] address the flexible standard of object-centric process data by introducing *gOCED*, which grounds the Object-Centric Event Data (OCED) Core Model in concepts from the UFO foundational ontology [12]. While *gOCED* provides a robust conceptual framework including event mereology and dynamic participation relationships, it remains a conceptual meta-modelling framework. Their work currently lacks a concrete demonstration for solving data-driven process mining problems via reasoning. For instance, while the `gufo:historicallyDependsOn` relation provides precise semantics, it is unclear how it might be used to reason over real-world, large-scale data.

Consequently, a critical gap remains: object-centric process mining methods lack a means to apply the robust semantics of foundational ontologies while providing tractable data-driven mechanisms to provide concrete answers to real-world problems.

3 Beyond Synchronization: Object Interaction Graphs

Beginning from an object-centric event log, we construct an Object Interaction Graph (OIG) that captures interactions between objects through events. We introduce a running example of a logistics company, where shipping containers are ordered, packages are loaded into those containers, containers are loaded onto trucks, forklifts unload the containers, and finally the containers are loaded onto a departure vehicle ¹. A snippet of the event log is given in table 1.

Our work starts from a simplified object-centric event log [24, 7]. Let $\mathcal{U}_E$ be the universe of events, $\mathcal{U}_A$ the universe of activities, $\mathcal{U}_O$ of objects, and $\mathcal{U}_{OT}$

¹ The example is based on ocel-standard.org/event-logs/simulations/logistics

Table 1. A snippet of an object-centric event log for logistics tracking

Event ID	Activity	Timestamp	Involved Objects
e_1	Order Containers	08:00	c_1, c_2
e_2	Load onto Truck	08:30	c_1, t_1
e_3	Load Packages	09:00	c_1, t_1, p_1, p_2
e_4	Load onto Truck	09:30	c_2, t_1
e_5	Load Packages	10:00	c_2, t_1, p_3
e_6	Unload Container	12:00	c_1, t_1, f_1
e_7	Unload Container	15:00	c_2, t_1, f_2
e_8	Load Departure Vehicle	17:30	c_1, c_2, f_1, v_1

of object types. We define a minimal² object-centric event log as a tuple $L = (E, O, OT, act, obj, otype, \prec)$ as follows, similar to Van Der Aalst et al. [24]:

$E \subseteq \mathcal{U}_E$	a finite set of events
$O \subseteq \mathcal{U}_O$	a finite set of objects
$OT \subseteq \mathcal{U}_{OT}$	a finite set of object types
$act \subseteq E \times \mathcal{U}_A$	maps an event to an activity
$obj \subseteq E \times \mathcal{P}(O)$	maps an event to participating object(s)
$otype \subseteq O \times OT$	maps an object to a unique type
$\prec \subseteq E \times E$	a total order on E

For each object $o \in O$, we define its object trace³ as the unique ordered sequence of events involving o induced by $\prec$:

$$Tr(o) = \langle e_1, \dots, e_k \rangle \quad \text{such that} \quad e_i \in E \wedge o \in obj(e_i) \wedge e_i \prec e_{i+1} \quad (1 \leq i < k).$$

We denote their start and end events as $start_{tr}(o) \in E$ and $end_{tr}(o) \in E$.

For the logistics running example, container c_1 's object trace is given by:

$$Tr(c_1) = \langle e_1, e_2, e_3, e_6, e_8 \rangle.$$

The way in which these object traces interact will be particularly relevant to our methodology. I.e., they may intersect at common events, which is commonly referred to in process mining literature as “synchronization” [10, 14, 7]. We extract object synchronization relations $SR \subseteq \mathcal{P}(O) \times E$ for an event log L as follows:

$$SR = \{ \langle obj(e), e \rangle \mid e \in E \}$$

These SR thus encode an object model based on “synchronizations”. By further characterizing these object synchronizations, we can infer additional semantics for SR . Consider a synchronization of object o with other objects $\{o_i\}$ within event e , where e is the initial event for o_i ($e = start_{tr}(o_i)$), but an intermediary event for o . This describes a situation where objects o_i did not exist before event e , but object o did. Hence, it could be read as object o being involved in the

² I.e., any superset of this event log, such as with attributes, may also be used.

³ For instance, this is referred to as the Directly-Follows (DF) path by Fahland [7].

“creation” of objects o_i in event e [7]. For instance, in event e_3 , objects c_1, t_1 could be seen as being involved in the creation of objects p_1, p_2 .

Two critical caveats apply to these characterizations. First, event logs are often incomplete, meaning that object creation or closure may actually lie outside the data extraction window [7]. Second, the “creation” and “closing” terminology may be misleading, and, on the individual level, they are not that meaningful. From the prior example, a container c_1 and truck t_1 participating in an event where packages p_1, p_2 first appeared does not imply that c_1 and t_1 somehow “created” the packages⁴. Moreover, it can certainly not be inferred that, in general, containers and trucks create packages. Rather, it simply means that the individual c_1 and t_1 objects interacted with the packages as described above. An aggregate analysis, across object types and activities, can help validate whether these interactions are anecdotal or not; and which particular ontological category they represent, such as creation, closure, or some other category. This observation motivates our aggregated analysis in Sections 4 and 5, where patterns across multiple instances and object types reveal the underlying semantics, ultimately producing an inferred Object Role Ontology (ORO).

4 Aggregating and Interpreting the Object Interaction Graph

Aggregating synchronization relations at the schema level will better reflect the true nature of object interactions through process execution, and is a first step to inferring appropriate ontological categories.

We aggregate the object model from the prior section into an Aggregated Object Graph (AOG). For that purpose, we define:

$AC = \{act(e) \mid e \in E\}$	set of all activities
$SR_{ac}(a) = \{sr \in SR \mid sr = (O, a)\}$	synchronization relations of activity a
$OT_o(O) = \{otype(o) \mid o \in O\}$	object types of set O
$OT_{ac}(a) = \{OT_o(O) \mid (O, a) \in SR_{ac}(a)\}$	object types in activity a ’s synchronizations
$Group(O, OT) = \{O_i \mid o_{ti} \in OT,$	grouping object sets by type
$O_i = \{o \in O \mid otype(o) = o_{ti}\}$	

Then, $AR_{ac} \subseteq \mathcal{P}(\mathcal{P}(O)) \times AC$ represents the set of aggregated object relations for an event log L , alongside the activities of the synchronizing events:

$$AR_{ac} = \{(OG_a, a) \mid a \in AC\}.$$

Where, for each activity a , we obtain OG_{a_i} by collecting all synchronization relations $(O_{a_i}, a) \in SR_{ac}(a)$ and grouping each O_{a_i} by the object types for a :

$$OG_{a_i} = Group(O_{a_i}, OT_{ac}(a)), \quad OG_a = \{OG_{a_i}\}.$$

⁴ There are examples where the terminology is more fitting, such as an order being involved in creating an invoice [7].

For example, in our logistics scenario:

$$\begin{aligned} OG_{\text{Load Packages}} &= \{\langle \{c_1\}, \{t_1\}, \{p_1, p_2\} \rangle, \langle \{c_2\}, \{t_1\}, \{p_3\} \rangle, \dots\}, \\ OG_{\text{Unload Container}} &= \{\langle \{c_1\}, \{t_1\}, \{f_1\} \rangle, \langle \{c_2\}, \{t_1\}, \{f_2\} \rangle, \dots\}. \end{aligned}$$

We define a projection function π_{OT} that filters a grouped tuple $g = \langle O_1, \dots, O_n \rangle$ to retain only sets of object types $OT \subseteq \mathcal{U}_{OT}$:

$$\pi_{OT}(g) = \langle O_i \in g \mid \forall o \in O_i, otype(o) \in OT \rangle, \quad \pi_{OT}(OG_a) = \{\pi_{OT}(g) \mid g \in OG_a\}.$$

For instance, if we are specifically interested in how forklifts and containers interact through unloading events, we may use the following projection:

$$OG_{\text{Unload Container}}^{\{C, F\}} = \{\langle \{c_1\}, \{f_1\} \rangle, \langle \{c_2\}, \{f_2\} \rangle, \dots\}.$$

These aggregated object groups illustrate more accurately how objects interact and participate within a process. Firstly, a forklift handles a single container at a time, and a container cannot be lifted simultaneously by multiple forklifts. E.g., such a one-to-one constraint could also be found in a document review process, where a review activity requires exactly one reviewer to evaluate exactly one document. Secondly, even if an “unload container” event is, anecdotally, the first along a forklift’s object trace, it should not be interpreted as the container *creating* the forklift. Across the process’s object groups, the forklift will be found to participate in many subsequent instances of lifting containers; it therefore cannot be a creation event, since an entity cannot be created multiple times.

Based on our defined projection functions and object groups, we map the properties of these sets to foundational ontological roles (Section 5), formalizing how objects interact and participate in a broader process.

5 Object Role Ontologies: Formalization in $\mathcal{ALCQI}$

To capture the ontological relationships implicit in aggregated object interaction graphs (AOGs), we first formalize these structures into the *Object Role Ontology* (ORO). The ORO characterizes the roles that objects play within activities through events. Expressed in the description logic $\mathcal{ALCQI}$, the ORO uses reified object roles to represent information from the AOG. We adopt $\mathcal{ALCQI}$ as it offers a recognized balance of expressive power and computational tractability (EXPTIME-complete) [8]. Our definition of reified roles aligns with established practices in applied ontology, facilitating future alignment with prominent foundational ontologies including UFO [12], DOLCE [4], and BFO [18].

To construct the ORO from the AOG, we define the following concepts:

O_{ot} for each $ot \in OT$	Object type concepts, with $o : O_{ot} \iff otype(o) = ot$
$O \equiv \bigsqcup_{ot \in OT} O_{ot}$	Subsumes all O_{ot} concepts
$R_a(ot)$ for each $a \in AC$	Role concepts of ot in a , with individuals $r_{o,e}$ (see below)
$Role(ot) \equiv \bigsqcup_{a \in AC} R_a(ot)$	Subsumes all roles of object type ot
$Role \equiv \bigsqcup_{ot \in OT} Role(ot)$	Subsumes all role concepts

Playing Roles We introduce a relation to describe how objects are related to (or “play”) reified roles:

$$\begin{array}{ll} \text{playsRole} & \text{relation between objects and roles} \\ \exists \text{playsRole}.\top \sqsubseteq O & \text{domain restriction (objects only)} \\ \top \sqsubseteq \forall \text{playsRole}.\text{Role} & \text{range restriction (roles only)} \end{array}$$

A-box *playsRole* literals are populated by linking all objects o , occurring in an event e with activity a , and with type ot (OG_a^{ot}), to a role $r_{o,e} : R_a(ot)$ via *playsRole*. E.g., the role of a forklift f_1 in a load container (LC) event e_1 :

$$(f_1, r_{f_1, e_1}) : \text{playsRole}, r_{f_1, e_1} : R_{LC}(F)$$

Events and Role Participation To better understand the ways in which different object roles interact through events, we describe how roles participate in events:

$$\begin{array}{ll} Ev_a \text{ for each } a \in AC \text{ event concepts with } e : Ev_a \iff \text{activ}(e) = a & \\ Event \equiv \bigsqcup_{a \in AC} Ev_a \text{ subsumes all events} & \\ participant & \text{relation between events and role instances} \end{array}$$

A-box *participant* literals are populated by linking corresponding events e to roles $r_{o,e}$ via *participant*. Both relationships are described by the axiom pattern:

$$R_a(ot) \sqsubseteq (= 1 \text{ participant}^- . Ev_a) \sqcap (= 1 \text{ playsRole}^- . ot)$$

I.e., a given role instance pertains to one event $e : Ev_a$ and one object $o : ot$.

Role Ordering We define the *subsequentRole* relation that captures temporal ordering in terms of object trace information:

$$\begin{array}{ll} \text{subsequentRole} & \text{ordering relation over role instances} \\ \exists \text{subsequentRole}.\top \sqsubseteq \text{Role} & \text{domain restriction (roles only)} \\ \top \sqsubseteq \forall \text{subsequentRole}.\text{Role} & \text{range restriction (roles only)} \\ \exists \text{subsequentRole}.\text{Role}(ot) \sqsubseteq \text{Role}(ot) & \text{object type preservation (domain)} \\ \text{Role}(ot) \sqsubseteq \forall \text{subsequentRole}.\text{Role}(ot) & \text{object type preservation (range)} \end{array}$$

Ordering is induced from traces by linking consecutive role instances:

$$r_{o, e_1} < r_{o, e_2} < \dots < r_{o, e_k}$$

where $<$ denotes the transitive closure subsequentRole^+ .

Since transitive closure is not expressible in $\mathcal{ALCQI}$, the literals are materialized in the ABox as a partial ordering via immediate successors:

$$(r_{o, e_i}, r_{o, e_{i+1}}) : \text{subsequentRole} \quad (1 \leq i < k)$$

Sub-roles We can now further characterize object roles as sub-roles based on their temporal ordering. For instance, we define the *init* and *end* sub-roles via the role ordering relation, *subsequentRole*, as:

$$\begin{array}{ll} r_{o, e} : R_{init} \iff r_{o, e} : \neg(\geq 1 \text{ subsequentRole}^{-1} . \text{Role}) \\ r_{o, e} : R_{end} \iff r_{o, e} : \neg(\geq 1 \text{ subsequentRole} . \text{Role}) \end{array}$$

I.e., a given role $r_{o, e}$ is an *init* role iff object o has no earlier roles, and it is an *end* role iff object o has no later roles.

5.1 Constraint Patterns

Based on these reified roles, their ordering, and their relationships within events, we can enhance the ORO with constraint patterns that help align with ontological categories. We express these patterns in $\mathcal{ALCQI}$ by specifying constraints on the reified role concepts.

For each role $R_a(ot)$ we define the following constraint patterns, based on values $min_a(ot)$, $max_a(ot)$, $min_a^{global}(ot)$ and $max_a^{global}(ot)$ from the AOG:

- **Per-instance cardinality.** The constraint on an event of Ev_a for activity a specifies how many $R_a(ot)$ roles participate in that event:

$$Ev_a \sqsubseteq (\geq min_a(ot) \text{ participant}.R_a(ot)) \sqcap (\leq max_a(ot) \text{ participant}.R_a(ot))$$

- **Global reusability.** The constraint on the object type specifies how many times objects of that type can play role $R_a(ot)$ across all events of a :

$$ot \sqsubseteq (\geq min^{global} \text{ playsRole}.R_a(ot)) \sqcap (\leq max^{global} \text{ playsRole}.R_a(ot))$$

If $max^{global} = 1$, the object plays the role exactly once (e.g., irreversible transformations, creation, closure). If unconstrained or > 1 , the object was found to be reusable in that role.

- **Cross-role constraints.** Restriction of the roles objects play across activities can be expressed by a subsumption in terms of these roles:

$$\exists \text{ playsRole}.R_a(ot) \sqsubseteq \exists \text{ playsRole}.R_{a'}(ot)$$

For instance, to say only containers that have been loaded can be unloaded can be expressed as: $\exists \text{ playsRole}.R_{unload}(C) \sqsubseteq \exists \text{ playsRole}.R_{load}(C)$

Logistics example. Returning to our logistics running example, we formalize the *Load Packages* activity (LP), where packages are loaded into containers on trucks. We introduce the reified role concepts $R_{LP}(P)$, $R_{LP}(C)$, and $R_{LP}(T)$ for packages, containers, and trucks respectively.

The per-instance participation constraints specify the *event-level behaviour*:

$$\begin{aligned} Ev_{LP} &\sqsubseteq (= 1 \text{ participant}.R_{LP}(C)) \\ Ev_{LP} &\sqsubseteq (= 1 \text{ participant}.R_{LP}(T)) \\ Ev_{LP} &\sqsubseteq (\geq 1 \text{ participant}.R_{LP}(P)) \sqcap (\leq 6 \text{ participant}.R_{LP}(P)) \end{aligned}$$

This expresses that each *Load Packages* event requires exactly one container and one truck, and between one and six packages.

The inverse global reusability constraints instead specify the *lifecycle behaviour* of the object types:

$$\begin{aligned} P &\sqsubseteq (= 1 \text{ playsRole}.R_{LP}(P)) \\ C &\sqsubseteq (\geq 1 \text{ playsRole}.R_{LP}(C)) \\ T &\sqsubseteq (\geq 1 \text{ playsRole}.R_{LP}(T)) \end{aligned}$$

These global constraints distinguish “consumption” or single-use roles (that of packages) from “resource-like” roles (that of containers and trucks). Notably, the properties of these object types are not defined at a global level but per activity. This allows for the possibility that some objects may be used as reusable resources in some roles, but may be consumed in other roles. For example, a wooden shipping pallet acts as a reusable transport resource during standard delivery events, but is consumed during recycling or disposal events.

5.2 Categorical Inference

Building on the ORO formalization and constraint patterns introduced above, object roles can be aligned with ontological categories that reflect the fundamental and fine-grained ways in which these objects interact.

Global initialization or ending We can infer a global initialization/ending for a role concept $R_a(ot)$ iff every instance of that Role concept is also an instance of the *init*/*end* sub-role. We define this behavior as $R_a(ot) \sqsubseteq R_X$, where X denotes *init* or *end*. Consider the example of forklift f_1 whose trace begins with:

$$start_{tr}(f_1) = unload_container$$

In this instance the role forklift f_1 plays is indeed an init role, identified by:

$$(f_1, r_{e_1, UC}) : playsRole, r_{e_1, UC} : R_{init}.$$

However, across the global process, we find that forklifts playing several non-init roles in *other* unload_container events. Since we cannot demonstrate $R_{UC}(F) \sqsubseteq R_{init}$, we can determine that the role forklifts play in unloading containers is *not* one where they are created by these containers.

Optional and Alternative Sub-roles We can infer cases where objects may be ended or initialized in alternative events, by combining sub-roles of *init* or *end* with global reusability. For instance:

$$R_{a_i}(ot) \sqsubseteq R_{end}, \quad ot \sqsubseteq (\leq 1 \text{ playsRole}.R_{a_i}(ot)) \quad \text{for } i \in \{1, 2\}$$

With this constraint, we can model and detect alternative/optional initializations and closures. We can further populate the ORO with more sophisticated role relationships such as transformations.

Transformation A one-to-one transformation occurs when an event of activity a marks the end of an input object’s lifecycle (ot_{in}) and the beginning of an output object’s lifecycle (ot_{out}). We define this behavior by coupling sub-role alignment with strict unity constraints through the following set of constraints:

$R_a(ot_{in}) \sqsubseteq R_{end}$	input roles terminate object lifecycles
$R_a(ot_{out}) \sqsubseteq R_{init}$	output roles initiate object lifecycles
$Ev_a \sqsubseteq (= 1 \text{ participant}.R_a(ot_{in}))$	exactly one input role participant
$Ev_a \sqsubseteq (= 1 \text{ participant}.R_a(ot_{out}))$	exactly one output role participant
$ot_{in} \sqsubseteq (= 1 \text{ playsRole}.R_a(ot_{in}))$	one-time lifecycle participation of input
$ot_{out} \sqsubseteq (= 1 \text{ playsRole}.R_a(ot_{out}))$	one-time lifecycle participation of output

Consider a **Convert** activity within a sales process, transforming a Quote (Q) into a Customer Order (CO). The R_{end} constraint ensures that a quote cannot be converted multiple times (preventing double-ordering), while R_{init} ensures that the order originates solely from that specific conversion event. By enforcing $max^{global} = 1$ for both objects, we further maintain a strict one-to-one lineage where the identity of the quote is ‘consumed’ to produce a unique order instance.

Resource Behaviour Some objects may take on roles which they repeat across the process, while others are singular. For instance, the items delivered to a customer will not be re-used in another order fulfillment, while the forklift that loaded the order will. We can define this behavior using global reusability constraints:

$$ot \sqsubseteq (\geq 0 \text{ playsRole}.R_a(ot)) \sqcap (\leq N \text{ playsRole}.R_a(ot)). \quad \text{where } N \geq 2$$

This specifies that ot behaves in a resource-like capacity for activity a . Furthermore, one may characterize the object more generally based on all the resource-like roles it plays. For instance, since the forklift is always reusable, and neither created nor destroyed across the logs, it can be considered an *external resource*.

This formalization aligns object roles with foundational role categories including creation, transformation, and closure, and object categories (based on all the roles they play) such as external resources. These definitions allow categories to be inferred inductively from data without user input. Conversely, the framework also supports a top-down approach, where users can specify the expected role or object category for objects or roles respectively from a predefined library. The system can then evaluate the log to identify all the counterexamples where objects or roles fail to satisfy their intended categories. The application of both approaches is demonstrated in Section 6.

5.3 Application and Extension of Object Role Ontologies

The foundation of trace-aware object role and participation relations, together with constraint patterns, enables a highly extensible ontology of object interaction categories. As demonstrated by the one-to-one transformation category, which evaluates sub-roles governed by strict global and per-instance constraints, the ORO enables the formal inference of complex semantic categories.

While in this paper, we focused on salient examples of ontological categories to enrich the ORO, many other categorizations can be defined. By further combining sub-role interactions and cross-role constraints, the ORO framework can capture highly complex process behaviours. For instance, differing cardinality constraints on the input or output side of the transformation pattern can detect *component splits* or *batching* behaviours; combining cross-role constraints with temporal ordering enables the detection of *containment* or *control* behaviours. For example, we can determine that a truck contains and controls a specific set of packages if the packages loaded onto it are a superset of those eventually unloaded from it across subsequent events.

6 Validation and Demonstration

To implement the proposed pipeline of raw object-centric data into an Object Role Ontology (ORO), we utilize a performant and multi-stage pipeline. For the construction and management of the Object Interaction Graph (OIG) and Aggregated Object Graph (AOG), we employ DuckDB [21] leveraging its demonstrated efficiency in handling the complex joins and aggregations required for object-centric event analysis [15]. The mapping into description logic to construct the ORO is subsequently handled with a Virtual Knowledge Graph (VKG) layer, implemented with the *ontop* [2] framework. This virtualization approach maintains data within a performant analytical environment, while exposing it to the expressive DL-based querying capabilities of *ontop*.

While traditional VKG applications in process mining [22, 5] focus on data access and extraction, our approach facilitates both ontology induction and automated verification. To achieve this, we leverage *ontop* for two primary functions. First, bottom-up induction identifies *ALCQI* constraint patterns to automatically enhance the ORO. Second, top-down verification employs parametric queries to detect counter-examples where log behaviour deviates from formal domain assumptions. These components are integrated via a Python library that serves as a functional front-end. By wrapping both the analytical power of DuckDB and the SPARQL-driven reasoning of the VKG, the library provides a practical, programmatic interface. This allows users to construct an ORO and visualize its roles within a few lines of code⁵.

6.1 Real-World Application and Results

To demonstrate this methodology, we applied it to several standard and real-world OCEL datasets⁶. We focus here on our application to an OCEL 2.0 adapted BPI Challenge 2017 dataset [20]. Generation of the BPI2017 OIG and AOG on a test machine (Ryzen 7 255, 32GB DDR5) ran in 3.47s, with role SPARQL queries ranging from 0.063 to 0.28 seconds. Figure 1 and 2 depict demonstrative selections of inferred roles and their constraints, used to form an object role ontology (ORO).

These properties align the **resource** to the *external resource* role in the discovered ORO, in line with existing domain knowledge.

$$\begin{aligned} \text{Resource} &\sqsubseteq (\geq 0 \text{ playsRole.R}_{\text{Submitted}}(\text{Resource})) \\ &\sqcap (\leq 994 \text{ playsRole.R}_{\text{Submitted}}(\text{Resource})). \end{aligned}$$

Meaning any given resource may perform this role between 0 and 994 times; they are globally re-useable. Additionally, the individual roles played are not all

⁵ The preliminary implementation relies on Docker containers for simplified deployment and is available at <https://anonymous.4open.science/r/oro-demo/>

⁶ The complete ORO construction and visualizations of BPI2017 and three other benchmark datasets is available at:
https://anonymous.4open.science/r/oro-demo/ocel_o2o_inference.ipynb

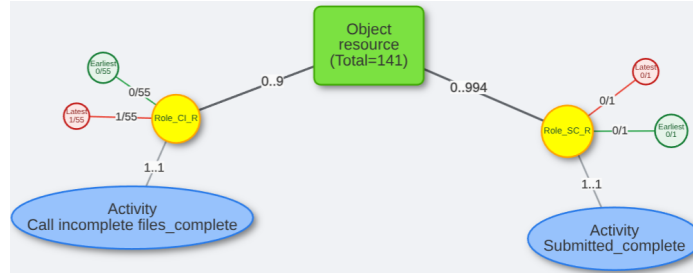


Fig. 1. Selection of Inferred Role Properties for Resource Object Type

instances of the *Role_{init}* sub-role (in fact they never are). These same behavioural properties are true for every role⁷ played by the **resource** object, meaning it is re-usable in every observed role, and has no consistent initialization or ending within the system, so it aligns to an *external resource* in the ORO. Conversely, the inferred role depicted in Figure 2 results in an *optional closure*.

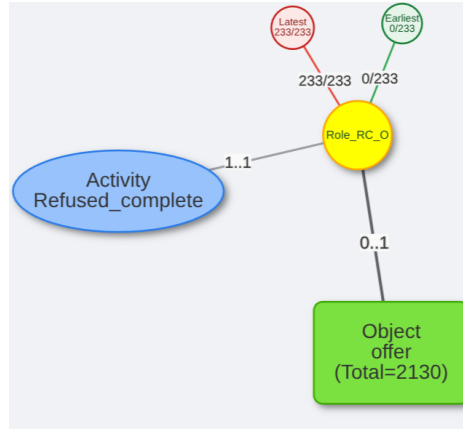


Fig. 2. Inferred Role Properties for Resource Object Type

6.2 Extensions

Rather than employ fuzzy logic, thresholds or tolerances parameters, our methodology utilizes crisp formal logic to allow practitioners to carefully consider their

⁷ Due to the large volume of activities in this real-world log, we do not show all roles here, but the full analysis is available at:

https://anonymous.4open.science/r/oro-demo/ocel_o2o_inference.ipynb

data and how it matches up to their implicit domain understanding. In an idealized setting with complete and noise-free data, the inductively generated model provides an accurate representation of the process structure. In practice, however, the strictness of the top-down verification stage serves as an important analytical counterbalance. It allows practitioners to identify discrepancies between the inductive model and their intended semantics of the domain. Moreover, in response to such discrepancies, one can for instance (a) correct process execution (e.g. retrain staff to following the correct procedure), (b) update data logging methods to capture missing events, or (c) refine one’s domain understanding based on what the bottom-up analysis reveals.

The framework also establishes a foundation for semi-online verification. Once the initial object role ontology is inferred, refined and validated using a top-down approach, the resulting ontology can be deployed to verify incoming log entries. For instance, if a `customer order` object is later observed undergoing a second creation event, this would result in an inconsistency with the ORO. In this way, the system supports continuous conformance checking between the evolving event log and the expected ontological commitments. More broadly, this approach improves accessibility for practitioners by aligning raw event data with interpretable ontological categories. It also creates opportunities for constraint reuse across systems that share common object types and interaction patterns.

7 Future Work, Discussion, and Conclusion

We presented a methodology to distill an object role ontology (ORO) from an object-centric event log. By abstracting the object synchronization behavior in a process, the ORO allows deriving the ontological grounding for the roles these objects play; e.g., creation, containment, resource utilization, transformation, and closure. A bottom-up modality can infer the ORO implied by the event log, while a top-down modality allows verifying existing domain assumptions.

While the methodology presented opens up new ways to analyze object-centric process data, the current approach has clear limitations that outline our immediate future work. The bottom-up stage carries an inductive bias, and although the top-down stage injects domain knowledge, reasoning remains constrained to the contents of the event log. The use of fuzzy reasoning here could enhance the bottom-up stage for noisy logs. Moreover, a key next step involves supporting richer constraints patterns, additional external domain knowledge types, and object attributes, which would enable participation rules tied to object state.

Beyond these enhancements, establishing soundness and completeness between the specification and implementation is a central priority for this research program. To support this broader agenda and strengthen semantic interoperability, future work will focus on formal alignment of object role ontologies with verified foundational ontologies [11, 4, 12, 18]. Furthermore, capturing the ontological commitments implicit in user corrections to logs offers a promising avenue to refine these alignments enhance the top-down approach [17].

Ultimately, this paper contributes a robust, data-driven pathway from object-centric logs to formal ontological interpretation. By lifting object synchronizations into object interaction graphs, aggregating them, and formalizing them as object role ontologies expressed in *ALCQT*—all while leveraging virtual knowledge graphs directly on RDF/DL with SPARQL checks—the implementation remains tightly bound to the formal model. This work not only advances the structural analysis of event logs but establishes a critical, formal bridge between the fields of process mining and applied ontology.

References

1. Angles, R., Bonifati, A., Dumbrava, S., Fletcher, G., Green, A., Hidders, J., Li, B., Libkin, L., Marsault, V., Martens, W., et al.: Pg-schema: Schemas for property graphs. *Proceedings of the ACM on Management of Data* **1**(2), 1–25 (2023)
2. Bagosi, T., Calvanese, D., Hardi, J., Komla-Ebri, S., Lanti, D., Rezk, M., Rodriguez-Muro, M., Slusnys, M., Xiao, G.: The ontop framework for ontology based data access. In: *The Semantic Web and Web Science: 8th Chinese Conference, CSWS 2014, Wuhan, China, August 8–12, 2014, Revised Selected Papers* 8. pp. 67–77. Springer (2014)
3. Berti, A., Koren, I., Adams, J.N., Park, G., Knopp, B., Graves, N., Rafiei, M., Liß, L., Unterberg, L.T.G., Zhang, Y., et al.: Ocel (object-centric event log) 2.0 specification. *arXiv preprint arXiv:2403.01975* (2024)
4. Borgo, S., Ferrario, R., Gangemi, A., Guarino, N., Masolo, C., Porello, D., Sanfilippo, E.M., Vieu, L.: Dolce: A descriptive ontology for linguistic and cognitive engineering. *Applied ontology* **17**(1), 45–69 (2022)
5. Calvanese, D., Kalayci, T.E., Montali, M., Tinella, S.: Ontology-based data access for extracting event logs from legacy data: the onprom tool and methodology. In: *International Conference on Business Information Systems*. pp. 220–236. Springer (2017)
6. Consortium, W.W.W., et al.: Owl 2 web ontology language document overview (2012)
7. Fahland, D.: Process mining over multiple behavioral dimensions with event knowledge graphs. In: *Process mining handbook*, pp. 274–319. Springer (2022)
8. Franconi, E., Mosca, A., Solomakhin, D.: Orm2: formalisation and encoding in owl2. In: *OTM Confederated International Conferences" On the Move to Meaningful Internet Systems"*. pp. 368–378. Springer (2012)
9. Ghahfarokhi, A.F., Park, G., Berti, A., van der Aalst, W.M.: Ocel: A standard for object-centric event logs. In: *European Conference on Advances in Databases and Information Systems*. pp. 169–175. Springer (2021)
10. Gianola, A., Montali, M., Winkler, S.: Object-centric conformance alignments with synchronization. In: *International conference on advanced information systems engineering*. pp. 3–19. Springer (2024)
11. Grüninger, M.: Ontology of the process specification language. In: *Handbook on ontologies*, pp. 575–592. Springer (2004)
12. Guizzardi, G., Botti Benevides, A., Fonseca, C.M., Porello, D., Almeida, J.P.A., Prince Sales, T.: Ufo: Unified foundational ontology. *Applied ontology* **17**(1), 167–210 (2022)

13. Hooshyar, H., Fumagalli, M., Montali, M., Guizzardi, G.: Time and relations into focus: Ontological foundations of object-centric event data. *arXiv preprint arXiv:2512.14425* (2025)
14. Küsters, A., van der Aalst, W.M.: Oc-declare: Discovering object-centric declarative patterns with synchronization. In: *International Conference on Business Process Management*. pp. 162–179. Springer (2025)
15. Küsters, A., van der Aalst, W.M.: Ocpq: Object-centric process querying and constraints. In: *International Conference on Research Challenges in Information Science*. pp. 383–400. Springer (2025)
16. Moher, R., Gruninger, M.: Mining for meaning: Ontology-aware process mining methods through knowledge patterns. In: *International Conference on Research Challenges in Information Science*. Springer (2025)
17. Moher, R., Gruninger, M.: Process representation meets operational realization: An architecture for data-driven process ontology application through process mining. In: *Formal Ontology in Information Systems*. pp. 196–209. IOS Press (2025)
18. Otte, J.N., Beverley, J., Ruttenberg, A.: Bfo: Basic formal ontology. *Applied ontology* **17**(1), 17–43 (2022)
19. Pan, J.Z.: Resource description framework. In: *Handbook on ontologies*, pp. 71–90. Springer (2009)
20. Povalyaeva, E., Khamitov, I., Fomenko, A.: Bpic 2017: density analysis of the interaction with clients. *BPI Challenge* **2017** (2017)
21. Raasveldt, M., Mühleisen, H.: Duckdb: an embeddable analytical database. In: *Proceedings of the 2019 international conference on management of data*. pp. 1981–1984 (2019)
22. Swevels, A., Fahland, D., Montali, M.: Implementing object-centric event data models in event knowledge graphs. In: *International Conference on Process Mining*. pp. 431–443. Springer (2023)
23. Van Der Aalst, W., Adriansyah, A., De Medeiros, A.K.A., Arcieri, F., Baier, T., Blickle, T., Bose, J.C., Van Den Brand, P., Brandtjen, R., Buijs, J., et al.: Process mining manifesto. In: *Business Process Management Workshops: BPM 2011 International Workshops, Clermont-Ferrand, France, August 29, 2011, Revised Selected Papers, Part I* 9. pp. 169–194. Springer (2012)
24. Van Der Aalst, W.M., Li, G., Montali, M.: Object-centric behavioral constraints. *arXiv preprint arXiv:1703.05740* (2017)
25. Van Woensel, W.: Process trace querying using knowledge graphs and notation3. *arXiv preprint arXiv:2409.04452* (2024)